# 04 · Alerts, notifications and badges

Personal alerts, community advisories, the citizen's visits page, the worker's notifications, and the unread badges in the menu.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

## What this feature does

Alerts are rows in the `alerts` table, written by other features: a flagged reading (guide 01), a booked, moved, cancelled or missed visit (guide 09). Citizens see them on **Alerts & visits**; workers see theirs on **Notifications**.

Community advisories are alerts with no owner (`user_id` empty); they are shown to everyone for 30 days and never count as unread.

The menu shows badges: unread alerts for citizens; for workers, unread notifications and the number of citizens with open flags. They refresh every 60 seconds and when you change page.

## Files involved

Only the files this feature uses.

```
backend/app/
├── api/routes/alerts.py          GET /alerts, GET /alerts/unread-count, POST /alerts/mark-read,
│                                 GET /phc-alerts/unread-count (worker badge)
├── services/visits.py            writes the visit alerts (booked, moved, cancelled, missed)
└── models/alert.py               table alerts
frontend/src/
├── pages/citizen/Alerts.tsx      "Alerts & visits" for citizens
├── pages/Notifications.tsx       the worker's notifications
├── components/app/AlertList.tsx  the list, shared by both pages; marks shown alerts read
└── components/app/AppShell.tsx   useBadges(): the menu counters
```

## How the pieces connect

1. A feature writes an `Alert` row (for example `notify_booked` when a visit is booked).
2. `AppShell` polls the unread count, so the badge shows a number.
3. The person opens the alerts page; `GET /alerts` returns their alerts plus recent advisories.
4. `AlertList` marks the unread ones as read (`POST /alerts/mark-read`); the badge goes back to zero on the next check.

## Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

### 1. `backend/app/services/visits.py` · notify_booked

An example of where alerts come from: one booking writes two alerts, worded for each reader.

```
22  def notify_booked(session: AsyncSession, visit: ScheduledVisit, citizen: User, worker: User) -> None:
23      when = format_local(visit.scheduled_for)
24      session.add_all(
25          [
26              _alert(
27                  citizen.id,
28                  f"{worker.name} from your health centre will visit you on {when}"
29                  + (f" ({visit.reason})." if visit.reason else "."),
30                  AlertSeverity.medium,
31              ),
32              _alert(worker.id, f"Visit booked for {citizen.name} on {when}."),
33          ]
34      )
```

**What the code means**

- `session.add_all([...])` stages several rows at once; the caller commits.
- The citizen reads "... will visit you on Tue 24 Sep, 10:30 AM"; the worker reads the citizen's name.
- `(f" ({visit.reason})." if visit.reason else ".")` adds the reason only when there is one.

### 2. `backend/app/api/routes/alerts.py` · list_alerts

```
@router.get("", response_model=AlertsOut)
async def list_alerts(
    limit: int = Query(50, ge=1, le=200),
    unread_only: bool = False,
    user: User = Depends(require_any_verified),
    db: AsyncSession = Depends(get_session),
) -> AlertsOut:
    own = select(Alert).where(Alert.user_id == user.id)
    if unread_only:
        own = own.where(Alert.is_read.is_(False))
    alerts = (await db.scalars(own.order_by(Alert.created_at.desc()).limit(limit))).all()
    # Advisories have no owner, so they have no per-person read state and never count toward the badge.
    advisories = (
        await db.scalars(
            select(Alert)
            .where(
                Alert.alert_type == AlertType.community,
                Alert.user_id.is_(None),
                Alert.created_at >= now_utc() - ADVISORY_WINDOW,
            )
            .order_by(Alert.created_at.desc())
            .limit(ADVISORY_LIMIT)
        )
    ).all()
    return AlertsOut(
        alerts=[alert_out(a) for a in alerts],
        advisories=[alert_out(a) for a in advisories],
        unread=await _unread(db, user),
    )
```

#### What the code means

- `require_any_verified` allows citizens and workers alike: everyone has their own alerts.
- `own` is the query for the caller's alerts; `unread_only` narrows it if asked.
- Advisories: `Alert.user_id.is_(None)` (no owner) and created in the last 30 days (`ADVISORY_WINDOW`).
- `[alert_out(a) for a in alerts]` turns each row into the response shape.

### 3. `backend/app/api/routes/alerts.py` · mark_read

```
@router.post("/mark-read", response_model=Updated)
async def mark_read(
    data: MarkRead, user: User = Depends(require_any_verified), db: AsyncSession = Depends(get_session)
) -> Updated:
    # user_id in the WHERE: ids belonging to anyone else, or to no one (advisories), match nothing.
    query = update(Alert).where(Alert.user_id == user.id, Alert.is_read.is_(False)).values(is_read=True)
    if data.ids is not None:
        query = query.where(Alert.id.in_(data.ids))
    result = await db.execute(query)
    await db.commit()
    return Updated(updated=result.rowcount or 0)
```

#### What the code means

- `update(Alert).where(...).values(is_read=True)` is one SQL UPDATE for many rows.
- Because the `where` includes `Alert.user_id == user.id`, ids that belong to someone else (or to nobody, like advisories) match nothing.
- `result.rowcount` is how many rows changed; it is returned to the page.

### 4. `backend/app/api/routes/alerts.py` · phc_unread_count

```python
@phc_router.get("/unread-count", response_model=PhcBadge)
async def phc_unread_count(
    worker: User = Depends(require_catchment_worker), db: AsyncSession = Depends(get_session)
) -> PhcBadge:
    flagged = await db.scalar(
        select(func.count(func.distinct(ProcessedData.user_id))).where(
            ProcessedData.user_id.in_(catchment_ids(worker)),
            ProcessedData.is_anomaly.is_(True),
            ProcessedData.resolution.is_(None),
        )
    )
    return PhcBadge(unread=await _unread(db, worker), citizens_flagged=flagged or 0)
```

**What the code means**

- `func.count(func.distinct(ProcessedData.user_id))` counts **people**, not flags: one citizen with five flags is one piece of work.
- `.in_(catchment_ids(worker))` limits it to the worker's own citizens (guide 05).

### 5. `frontend/src/components/app/AlertList.tsx` · AlertList

```tsx
export function AlertList({ alerts, advisories }: { alerts: AlertItem[]; advisories: AlertItem[] }) {
  const sent = useRef(false);
  useEffect(() => {
    const unread = alerts.filter((a) => !a.is_read).map((a) => a.id);
    if (unread.length && !sent.current) {
      sent.current = true;
      void markRead(unread).catch(() => undefined);
    }
  }, [alerts]);

  return (
    <div className="flex flex-col gap-6">
      <Card title="Alerts">
        {alerts.length === 0 ? (
          <Empty title="No alerts" />
        ) : (
          <ul className="flex flex-col gap-2">
            {alerts.map((a) => (
              <li key={a.id} className={cx("rounded-lg border border-line border-l-4 bg-surface p-3 text-sm"
                  , TONE[a.severity])}>
                <div className="flex flex-wrap items-baseline justify-between gap-2">
                  <p className="text-ink">{a.message}</p>
                  {!a.is_read && <span className="rounded-full bg-brand-soft px-2 text-xs font-semibold text
                      -brand">New</span>}
                </div>
                <p className="mt-1 text-xs text-ink-soft">{fmtDateTime(a.created_at)}</p>
              </li>
            ))}
... (21 more lines in the file: the advisories card follows)
```

**What the code means**

- `useRef(false)` remembers "already sent" without causing a re-draw, so mark-read is sent once per visit.
- `void markRead(unread).catch(() => undefined)`: fire and forget; a failure here is not worth an error message. `void` says the Promise is ignored on purpose.
- Unread alerts keep their "New" label for this visit even after they are marked read on the server.
- `TONE[a.severity]` picks the border colour from a lookup object.

## 6. `frontend/src/pages/citizen/Alerts.tsx` · CitizenAlerts

```tsx
export function CitizenAlerts() {
  const { data, error, reload } = useResource((signal) => Promise.all([getMySchedule(signal), getAlerts(signal)]), "alerts");
  if (error) return <ErrorNote message={error} onRetry={reload} />;
  if (!data) return <Loading />;
  const [visits, alerts] = data;

  const cutoff = Date.now() - RECENT_MISSED_DAYS * 86400_000;
  const recent = (v: Visit) => v.state !== "missed" || new Date(v.scheduled_for).getTime() >= cutoff;
  const byTime = (a: Visit, b: Visit) => a.scheduled_for.localeCompare(b.scheduled_for);
  const attention = visits.filter((v) => ATTENTION.includes(v.state) && recent(v)).sort(byTime);
  const upcoming = visits.filter((v) => UPCOMING.includes(v.state)).sort(byTime);
  const past = visits.filter((v) => !attention.includes(v) && !upcoming.includes(v));

  return (
    <>
      <PageHeader title="Alerts & visits" />
      <div className="flex flex-col gap-6">
        {attention.length > 0 && <Section title="Needs your attention" visits={attention} empty="" />}
        <Section title="Your upcoming visits" visits={upcoming} empty="No visits booked." />
        {past.length > 0 && <Section title="Past visits" visits={past} empty="" />}
        <AlertList alerts={alerts.alerts} advisories={alerts.advisories} />
      </div>
    </>
  );
}
```

### What the code means

- Loads visits and alerts together with `Promise.all`, then splits visits into sections by their `state`.
- `ATTENTION.includes(v.state)` checks membership in a list; missed visits older than 30 days are left out.
- `a.scheduled_for.localeCompare(b.scheduled_for)` sorts ISO date strings in time order.

## 7. `frontend/src/components/app/AppShell.tsx` · useBadges

A **custom hook**: a function whose name starts with `use` and that uses other hooks.

```tsx
function useBadges(role: string | undefined) {
  const [citizenUnread, setCitizenUnread] = useState(0);
  const [phc, setPhc] = useState({ unread: 0, citizens_flagged: 0 });
  const location = useLocation();

  useEffect(() => {
    if (!role) return;
    const controller = new AbortController();
    const load = () => {
      if (role === "user") getUnreadCount(controller.signal).then(setCitizenUnread).catch(() => undefined);
      if (role === "phc_worker") getPhcBadge(controller.signal).then(setPhc).catch(() => undefined);
    };
    load();
    const timer = window.setInterval(load, BADGE_POLL_MS);
    return () => {
      controller.abort();
      window.clearInterval(timer);
    };
    // Re-read on navigation: reading the alerts page clears the badge.
  }, [role, location.pathname]);
  return { citizenUnread, phc };
}
```

### What the code means

- `window.setInterval(load, BADGE_POLL_MS)` checks every 60 seconds.
- The function returned from `useEffect` cleans up: it cancels requests (`controller.abort()`) and stops the timer when the page changes or the user signs out.
- `[role, location.pathname]` re-runs it on every navigation, so opening the alerts page clears the badge straight away.

## Syntax used in this feature

| Code | What it means |
|---|---|
| `update(T).where(...).values(...)` | An SQL UPDATE built in Python. |
| `func.count(func.distinct(col))` | COUNT(DISTINCT col). |
| `useRef(false)` | A value that survives re-draws without causing one. |
| `setInterval / clearInterval` | Run a function repeatedly / stop it. |
| `AbortController` | Lets code cancel a request in flight. |